

Python - basics at a glance

Programming tool: Thonny ... already installed at Raspberry Pi

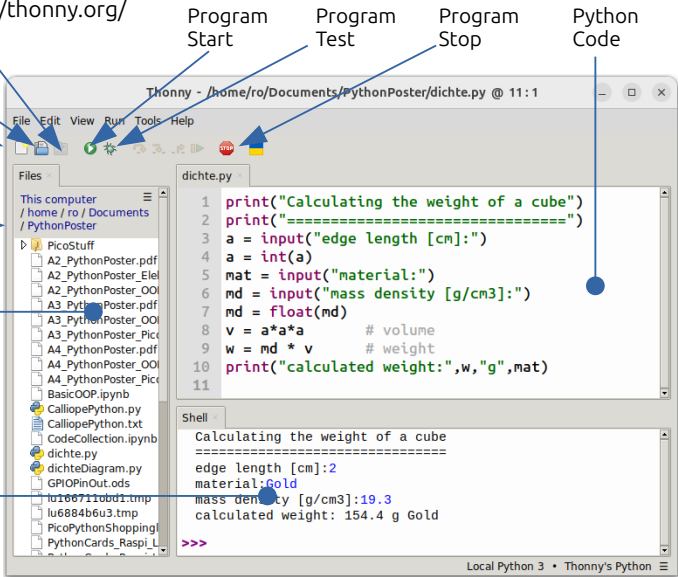
Download: <https://thonny.org/>

Save program
Load program
Create new program

Current directory

List of programs

Interactive shell for program output and command execution



. is used as decimal delimiter not comma
indicates the begin of code comment text

Note regarding if, else, while, for, def:
: colon must be written at the end of the statement
4 spaces are required to indent all related lines

1 Input, output, calculations

print will show text:
`print("Hello World")`

Variables are used to store values. This requires an assignment command:

`a = 10`

It is also possible to implement a calculation equation that will assign the result to a variable:

`w = md * v`

Current variable values can be shown with help of the print statement:
`print("Weight:", w)`

input passes keyboard input to a variable:
`text = input("How are you?")`

Every keyboard input is always a text string. In case numeric data is required, the text input needs to be converted explicitly to the desired numeric data type.
`a = input("edge length [cm]:")`
`a = int(a)`

`d = input("mass density [g/cm3]:")`
`d = float(d)`

int ... cast to integer number
float ... cast to floating point number

Calculating the weight of a golden cube

```
a = 10          # edge length cm
md = 19.3       # mass dens. Gold 19.3 g/cm3
v = a * a * a   # volume
w = md * v      # weight
print("Gold cube edge length", a, "cm")
print("weight:", w, "g")
```

```
print("Calculating the weight of a cube")
print("=====")
a = input("edge length [cm]:")
a = int(a)
mat = input("material:")
md = input("mass density [g/cm3]:")
md = float(md)
v = a*a*a       # volume
w = md * v      # weight
print("calculated weight:", w, "g", mat)
```

2 Conditional statements

if compares values and branches the program execution accordingly. If the comparing condition is met, the subsequent commands will be performed. These commands need to be indented by 4 spaces!

`if a>0:`
→ `v = a*a*a`
`print(v)`

else after an if contains commands to be performed in case the if-comparing-condition was not met:

`if a>0:`
→ `v = a*a*a`
`print(v)`
`else:`
→ `print("Invalid edge length.")`
`print("Program exit!")`
`exit(0)`

exit(0) ... terminates the program.

```
print("Calculating the weight of a cube")
print("=====")
a = input("edge length [cm]:")
a = int(a)
mat = input("material:")
md = input("mass density [g/cm3]:")
md = float(md)
if a>0 and md>0:
    v = a*a*a       # volume
    w = md * v      # weight
    print("weight:", w, "g", mat)
else:
    print("invalid values")
```

Notes on if-statements

An if- or else-command line needs to be terminated by a :

The coding of an else-part is not mandatory.

Multiple comparing conditions using and / or:
... and ... both conditions must be met
... or ... one matching condition is suitable

Potential comparisons:

< less than
> greater than
<= less or equal
>= greater or equal
== equal
!= not equal

3 Loops

while repeats commands as long as a comparing condition is met. All commands to be affected by the while-statement need to be indented by 4 spaces.

The following example illustrates the repeated execution of a calculation and print-out of the variable value as long as the value of a is less than 100. Each execution of the loop increments the value of a by 10.

```
a = 10
while a<100:
    v = a*a*a
    print(v)
    a=a+10
print("loop has been finished")
```

```
print("Calculating the weight of a cube")
print("=====")
a = input("edge length [cm]:")
a = input("start value:")
a = int(a)
b = input("end value:")
b = int(b)
mat = input("material:")
md = input("mass density [g/cm3]:")
md = float(md)
while a <= b:
    v = a*a*a       # volume
    w = md * v      # weight
    print(a, "cm ... weight:", w, "g", mat)
    a = a + 1
```

```
print("Calculating the weight of a cube")
print("=====")
mat = input("material:")
md = input("mass density [g/cm3]:")
md = float(md)
while True:
    a = input("edge length [cm]:")
    a = int(a)
    if a < 0:
        break
    v = a*a*a       # volume
    w = md * v      # weight
    print(a, "cm ... weight:", w, "g", mat)
    print("program finished")
```

while True: is used to implement a loop that runs, until it hits a **break** command to terminate the loop. An if-command is used to check a condition that is required to activate the break command.

In case the break command has stopped the loop, the program execution will continue with the 1st line after the loop.

4 Store data in lists

List

A list stores a sequence of values. All the values will be listed in squared brackets separated by comma.

A **for-in-loop** can be used to iterate all the different list items.

The **len**-function provides the number of list items.

The index is the position number of a list item. The first list item is located at index 0.

Use the **Index in squared brackets** to access a specific item of the list by its position number.

```
print("Calculating the weight of a cube")
print("=====")

material = ["Gold", "Silver", "Copper"]
massdens = [19.3, 10.5, 9.0]
```

```
print("-list of materials-")
for i in material:
    print(i)
```

```
a = input("edge length [cm]:")
a = int(a)
v = a*a*a       # volume
```

```
count = len(material)
index = 0
while index < count:
    mat = material[index]
    md = massdens[index]
    w = md * v      # weight
    print(a, "cm ... weight:", w, "g", mat)
    index = index + 1
print("program finished")
```

5 Store data in a dictionary

A **dictionary** stores pairs of **key : value**.

These pairs are written in curly brackets separated by comma.

The **keys()** statement provides the set of all the current keys.

Using the operation **element in set** one can check, whether a specific element is contained in a set or a list.

The **get(key)**-operation provides the value that is stored by the named key.

```
print("Calculating the weight of a cube")
print("=====")

massdens = {"Gold":19.3, "Silver":10.5, "Copper":9.0}
material = massdens.keys()
```

```
print("-list of materials-")
for i in material:
    print(i)
```

```
mat = input("material:")
if mat in material:
    a = input("edge length [cm]:")
    a = int(a)
    v = a*a*a
    md = massdens.get(mat)
    w = md * v      # weight
    print(a, "cm ... weight:", w, "g", mat)
else:
    print("material is not known")
```

6 Functions are used to provide a dedicated service

def ... begin of a function definition

function name

parameter

Functions are pieces of code implemented to provide a specific service. In many cases they are used to perform certain calculations or to show certain data.

A function has a name and it may have one or more parameters. All function related code must be indented by 4 spaces.

The **return**-statement is used to return the function result.

Variables that are defined within the function are only valid inside the function. Changes that have been performed on values of dictionaries or lists are visible outside the function after the function has been finished!

It is possible to call a function from all over a Python program.

```
def calculateWeight(edge, massdens):
    v = edge**3
    weight = massdens * v
    return weight
```

```
def showMaterials(materials):
    print("-list of materials-")
    for i in materials:
        print(i)
```

```
print("Calculating the weight of a cube")
print("=====")
```

```
massdens = {"Gold":19.3, "Silver":10.5, "Copper":9.0}
material = massdens.keys()
```

```
showMaterials(material)
mat = input("material:")
if mat in material:
    a = input("edge length [cm]:")
    a = int(a)
    md = massdens.get(mat)
    w = calculateWeight(a, md)
    print(a, "cm ... weight:", w, "g", mat)
```

7 Modules and Charts

Modules or libraries are collections of useful functions related to a certain topic. Modules that are required by a Python program need to be **imported** at the begin of the program. After this all functions of the imported modules are ready for use.

The **math**-module provides the sin, cos, and square-root functions (beside others).

The module **pyplot** provides services for chart plotting.

```
import matplotlib.pyplot as plt
```

```
def calculateWeight(edge, massdens):
    v = edge**3
    weight = massdens * v
    return weight
```

```
print("Calculating the weight of a cube")
print("=====")
```

```
massdens = {"Gold":19.3, "Silver":10.5, "Copper":9.0}
print(massdens.keys())
mat1 = input("Material 1:")
mat2 = input("Material 2:")
```

```
if mat1 not in massdens.keys():
    exit(0)
if mat2 not in massdens.keys():
    exit(0)
```

```
md1 = massdens.get(mat1)
md2 = massdens.get(mat2)
```

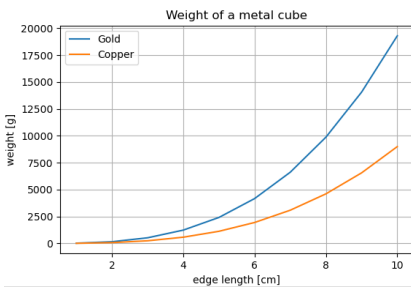
```
a = 1
weight1 = []
weight2 = []
edge = []
```

```
while a <= 10:
    w1 = calculateWeight(a, md1)
    w2 = calculateWeight(a, md2)
    weight1.append(w1)
    weight2.append(w2)
    edge.append(a)
    a = a + 1
```

create 3 empty lists

add data to the lists

```
Calculating the weight of a cube
=====
dict keys(['Gold', 'Silver', 'Copper'])
Material 1:Gold
Material 2:Copper
```



```
fig, ax = plt.subplots(figsize=(5,3))
ax.plot(edge, weight1, label=mat1)
ax.plot(edge, weight2, label=mat2)
ax.set_title("Weight of a metal cube")
ax.set_xlabel("edge length [cm]")
ax.set_ylabel("weight [g]")
ax.grid(True) # show grid
ax.legend() # show legend
plt.show()
```

plot numeric list values as a chart